实验 3. 基于贪心法的最小生成树算法

实验内容

本实验要求基于算法设计与分析的一般过程（亦即待求解问题的描述、算法设计、算法描述、算法正确性证明、算法分析、算法实现与测试），完成以下算法的设计与分析：**基于贪心法的最小生成树算法，包括 Prim 和 Kruskal 算法**。

实验目的

理解贪心法的核心思想以及求解过程。

实现语言

C++

实验结果

实验内容

基于贪心法，对无向连通带权图，分别用 **Prim** 和 **Kruskal** 算法求解**最小生成树（MST）**，输出所选边与总权值，并完成算法分析与实验总结。

问题描述

给定 **n 个顶点、m 条带权无向边**的连通图，求一棵**包含所有顶点、总权值最小、无环**的生成树，即最小生成树。

- 输入：顶点数、边集、各边权值
- 输出：总权值

算法设计

1. 算法策略
   - **Prim 算法**：从一个起点出发，每次选择**连接生成树与外部点的最小权边**，逐步扩展生成树（点贪心）。
   - **Kruskal 算法**：将所有边按权值升序，依次选边，用**并查集**判环，不成环则选中（边贪心）。
2. 数据结构
   - Prim：标记数组、最小权值数组
   - Kruskal：**边结构体**、排序、**并查集（路径压缩）**

算法描述

Prim 算法

函数 Prim(图 G, 起点 start):

    距离数组 dist[1...V] 全部设为 正无穷 (∞)

    访问数组 visited[1...V] 全部设为 false

```
    dist[start] = 0
    mst_weight = 0
    循环 V 次:
        u = -1
        min_dist = inf
        对于 每个节点 i 从 1 到 V:
            如果 visited[i] == false 且 dist[i] < min_dist:
                u = i
                min_dist = dist[i]
        如果 u == -1:
        visited[u] = true
        mst_weight = mst_weight + dist[u]
        对于 u 的每一条边 (u, v, weight):
            如果 visited[v] == false 且 weight < dist[v]:
                dist[v] = weight
    返回 mst_weight
```

Kruskal 算法

```
函数 Find(x):
    如果 parent[x] == x:
        返回 x
    parent[x] = Find(parent[x])
    返回 parent[x]
函数 Kruskal(节点数 V, 边集 Edges):
    将 Edges 按照 边权 (weight) 从小到大排序
    对于 每个节点 i 从 1 到 V:
        parent[i] = i
    mst_weight = 0
    edge_count = 0
    对于 Edges 中的每一条边 (u, v, weight):
        root_u = Find(u)
        root_v = Find(v)
        如果 root_u != root_v:
            parent[root_u] = root_v // 合并集合
            mst_weight = mst_weight + weight
            edge_count = edge_count + 1
    返回 mst_weight
```

算法复杂度分析

Prim 算法

- **时间复杂度**：$O(n^2)$
- **空间复杂度**：$O(n^2)$
- 适用：**稠密图**（边多、顶点少）

Kruskal 算法

- **时间复杂度**：O (E log E)（主要为排序开销）
- **空间复杂度**：O(E + n)

- 适用：**稀疏图**（边少）

代码如下图

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 2e5 + 10;
const int inf = 1e9;

int ans, n, m;
struct edge
{
    int v,w;
};
vector<edge>g[N];
void prim(int s)
{
    vector<int>vis(n + 1, 0), d(n + 1, inf);
    d[s] = 0;
    for(int i = 1; i <= n; i++)
    {
        int u = 0;
        for(int j = 1;j <= n; j++)
        if(!vis[j] && d[j] < d[u])u = j;
        vis[u] = 1;
        ans += d[u];
        for(auto [v, w] : g[u])
            if(d[v] > w)d[v] = w;
    }
    cout << "总权值为 " << ans << endl;
    ans = 0;
}

//队列优化
void queue_prim(int s){
    priority_queue<pair<int, int>, vector<pair<int,int>>, greater<>> que;
    vector<int>vis(n + 1, 0), d(n + 1, inf);
    d[s] = 0;que.push({0, s});
    while(que.size()){
        int u = que.top().second; que.pop();
        if(vis[u])continue;
        vis[u] = 1;
        ans += d[u];
        for(auto [v, w] : g[u]){
            if(d[v] > w)d[v] = w;
            que.push({d[v], v});
        }
    }
    cout << "总权值为 " << ans << endl;
    ans = 0;
}

//kruskal算法

struct node
{
    int u,v,w;
    bool operator<(const node &t)const
    {
        return w < t.w;
    }
}e[N];

int fa[N];
int find(int x)
{
    if(fa[x] == x)return x;
    return fa[x] = find(fa[x]);
}

void kruskal(){
    sort(e + 1, e + m + 1);
    for(int i = 1; i <= n; i++) fa[i] = i;
    for(int i = 1; i <= m; i++){
        int x = find(e[i].u), y = find(e[i].v);
        if(x != y){
            fa[x] = y;
            ans += e[i].w;
        }
    }
    cout << "总权值为 " << ans << endl;
    ans = 0;
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= m; i++){
        int u, v, w; cin >> u >> v >> w;
        e[i] = {u, v, w};
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    prim(1);
    queue_prim(1);
    kruskal();
}
```

运行结果 以及 时间比较如下图

下述为 **n = 70 m = 100** 的稀疏图

```
40 48 28
17 46 710
37 52 935
24 43 80
26 29 854
1 70 631
21 49 391
49 53 223
16 25 32
25 28 490
14 34 49
24 65 871
34 51 773
7 11 950
3 51 260
9 21 3
17 24 683
34 60 331
32 66 582
14 16 828
14 68 339
3 31 221
总权值为 24515
[Prim 算法] 运行耗时: 0.1 ms
总权值为 24515
[队列优化Prim 算法] 运行耗时: 0.07 ms
总权值为 24515
[Kruskal 算法] 运行耗时: 0.042 ms

请按任意键继续. . .
```

下述为 **n =70 m = 2000** 的稠密图

```
62 67 71
45 53 876
30 38 2
48 49 511
8 17 625
17 69 702
47 55 483
37 38 194
33 66 120
8 47 44
24 27 737
35 56 634
26 50 731
29 53 149
1 9 267
33 51 395
14 45 595
10 44 175
8 69 558
20 53 474
39 48 289
7 41 547
总权值为 1545
[Prim 算法] 运行耗时: 0.108 ms
总权值为 1545
[队列优化Prim 算法] 运行耗时: 1.03 ms
总权值为 1545
[Kruskal 算法] 运行耗时: 0.169 ms

请按任意键继续. . .
```

可以看到稀疏图 **kruskal** 更快 ， 而稠密图则 **Prim** 更快

队列优化在稀疏图上接近 **kruskal** 优于 **prim**

而在稠密图中 优先队列带来的常数开销使其慢于其他两者

实验总结

1. 遇到问题与解决

   Prim 初始化错误，导致权值异常。解决：从 1 号顶点开始，正确初始化 d 数组

2. 实验收获

   理解贪心 “**局部最优→全局最优**” 的核心。

   掌握 Prim 与 Kruskal 的原理、适用场景与实现。

熟练使用 C++ 实现图结构、排序、并查集。

3. 贪心法举一反三

贪心适用：**最优子结构**、**贪心选择性质**。

可扩展：哈夫曼编码、活动选择、单源最短路径 Dijkstra、分数背包等。

思路：先找**每一步最优规则**，再证明正确性，最后编码实现。